

华为云码道背后的 Coding Agent 系统设计

演讲人：张琦

华为技术有限公司 / 代码智能体技术专家

AiCon

全球人工智能开发与应用大会

目录

AI 辅助研发行业变革

Coding Agent 华为落地现状

Coding Agent 面临挑战

解构华为云码道

总结与展望

05

06

01 AI 辅助研发行业变革

辅助编程时代伊始



GitHub
Copilot



上下文感知补全

基于大语言模型分析代码上下文，精准预测并生成后续代码片段，让编码更流畅。



IDE深度集成与功能增强

无缝嵌入主流开发工具，不仅补全代码，更能生成函数、建议测试用例与解释代码功能。



全球用户规模

2000万+

覆盖全球数百万开发者群体



付费订阅用户

470万

高粘性用户持续增长



企业级客户数

7.7万+

企业采用率稳步攀升



财富100强采用率

90%

行业顶尖企业的共同选择

AI 辅助研发技术演进路径

2021-2023 · 函数级代码生成

基于Codex模型，实现基础的代码补全与片段生成，聚焦单一函数与代码块的辅助编写，解决局部编码效率问题。

2023-2024 · 强大的代码 LLM

GPT-4/5、CodeLlama等模型登场，具备更强的语义理解与复杂代码生成能力，开始支持多语言、多文件的协同编码。

2025-至今 · Agentic 框架兴起

LLM结合工具调用与自主规划，实现从代码生成到完整软件项目的自主构建、调试与维护，形成闭环的智能 workflows。

© 模型能力跃迁

从理解孤立代码片段，进化到深度理解复杂项目结构、依赖关系与业务逻辑。

🔍 上下文窗口扩展

窗口容量大幅提升，AI可“阅读”并处理整个代码库的上下文，实现全局视角的编码。

🔗 工具调用机制成熟

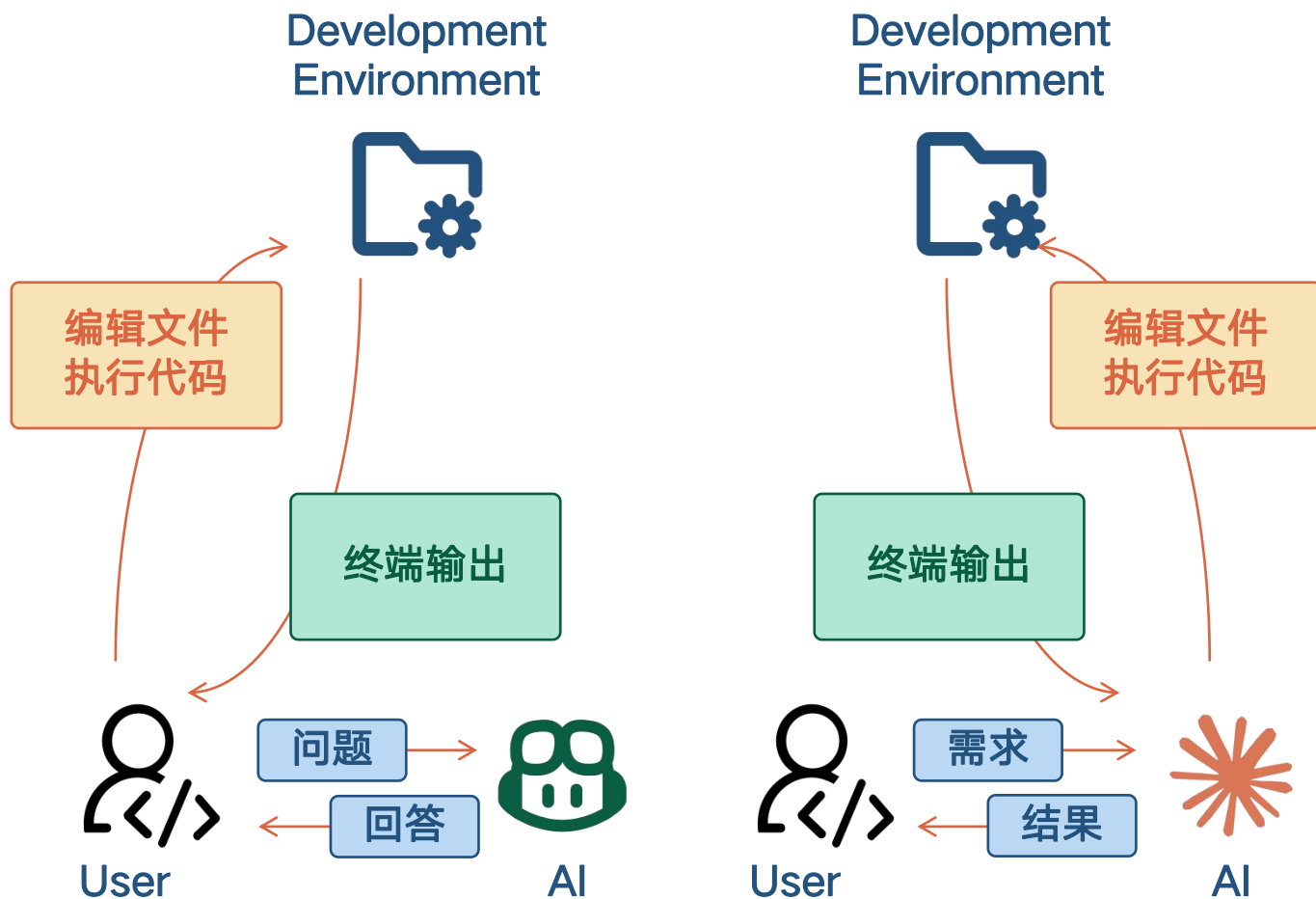
具备调用外部API、数据库、命令行工具的能力，将逻辑推理转化为对环境的实际操作。

⚙️ 自主规划与反思

形成Plan-Do-Check-Act闭环，能自主拆解任务、执行操作，并根据反馈迭代优化方案。



AI 的地位发生变化



被动响应 → 主动规划



Copilot 等待指令输入，是被动的“副驾驶”；Agent 接收高层任务目标，主动拆解步骤、规划路径并执行，成为积极的“执行者”

代码片段 → 项目级任务



从擅长生成单一函数或代码块，进阶为处理跨文件修改、环境配置、自动化测试与部署等完整的端到端项目任务。

工具使用者 → 智能体协作者



人机交互模式发生根本转变，开发者不再直接操纵工具，而是与具备自主决策能力的智能体进行目标对齐与进度协作，释放创造力于核心设计。

AI 辅助研发理念转变

Prompt Engineering

核心问题：如何通过语言指令引导黑盒模型输出预期结果

典型技术：系统提示词、Few-shot 示例、思维链 (CoT)、输出格式约束

代表产品：Copilot, Cursor Tab, CodeMate 代码续写

模型角色：高级计算器

Context Engineering

核心问题：如何设计模型的完整输入空间，使其获得任务所需的全部信息

典型技术：RAG 检索增强生成、结构化记忆系统、MCP (模型上下文协议)、长上下文窗口优化

代表产品：CodeMate 智能问答，其他的问答助手

模型角色：研究助理

Harness Engineering

核心问题：如何设计 Agent 的完整运行时环境，使其能够自主、可靠、持续地完成复杂任务

典型技术：执行编排，状态管理，结果验证，安全约束，错误恢复，多 Agent 协同

代表产品：Claude Code, Cursor Agent, CodeAgent

模型角色：自主员工

Harness Engineering 的本质：真正的工程挑战，从来都不是如何让 AI 做更多的事，而是如何让它在做错事的时候，不会造成不可挽回的损失。

问题一：Prompt Engineering 和 Context Engineering 消失了吗？

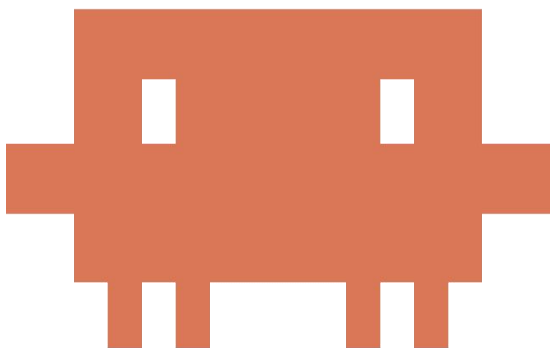
问题二：随着模型能力越来越强，还需要 Harness Engineering 吗？

02 Coding Agent 华为落地现状

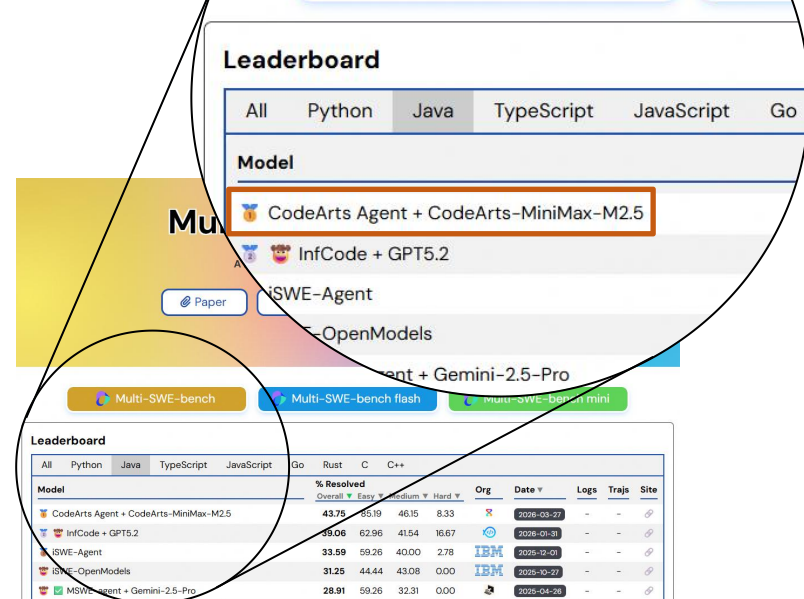
华为 Coding Agent 演化路径



CodeMate 内置开源工具 Cline,
对接 Qwen3 32B 模型



CodeMate 内置 Claude Code,
对接 GLM4.5 模型



CodeArts Agent 综合能力达到
Multi-SWE-bench Java 榜单 Top1

2025-05

2025-08

2025-12

2026-01

2026-04



CodeMate 上线第一版自研 CodeAgent



华为云码道（CodeArts）代码智能体公测

■ 案例: 安卓到鸿蒙, 10 天迁移 3 个 APP



阶段	关键技术
阶段一: 逆向代码为 Spec	基于多叉树的 Spec 生成技术
阶段二: Spec 生成 ArkTS	鸿蒙 Skills 生成与优化技术、多 Agent 动态编排技术等
阶段三: AI 辅助问题定界	端云协同 AI 问题定位定界与修复建议生成
阶段四: 质量保障	生成代码符合公司编码规范, 通过功能/DFX 测试
阶段五: 测试反馈闭环	基于 AI 的自动化测试用例生成、测试 Spec 自动生成等

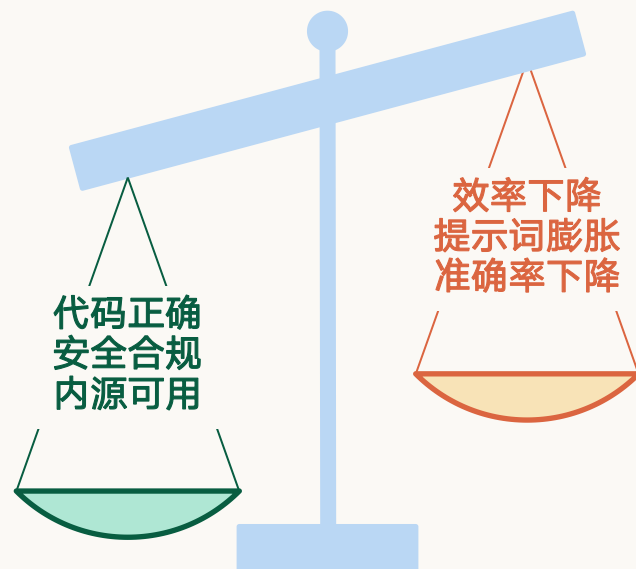


AntennaPod (11万代码)

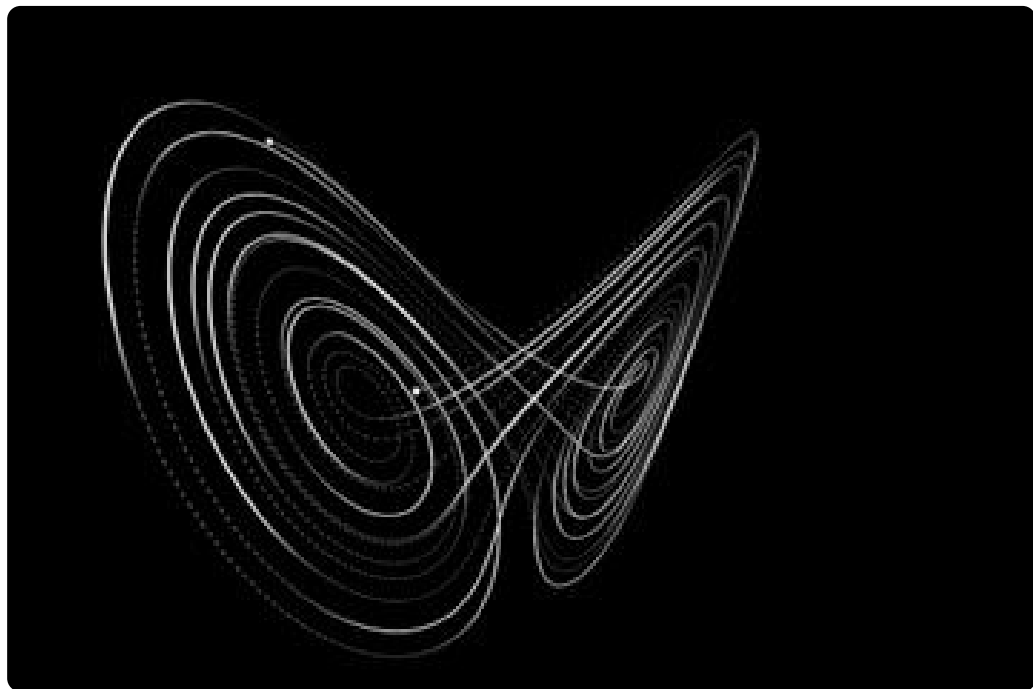
03 Coding Agent 面临挑战

挑战一：牵一发而动全身

- Bad Case 1: Agent 不知道内源地址, 导致无法安装依赖
 - 解决方案: 在系统提示词里写清楚如何安装内源依赖
 - 方案代价: pip, conda, poetry, mvn, vue, nodejs, c, c++ 每一种开发环境都不一样都需要写一遍, 提示词污染+膨胀, 挤占上下文窗口, 分散模型注意力, 影响其他效果. 同时硬编码在系统提示词里, 降低 Agent 通用性以及可维护性.
- Bad Case 2: Agent 生成的 Java 代码有错误 & Agent 生成的 Python 代码有错误
 - 解决方案: 在系统提示词里强调写完代码要编译运行执行测试
 - 方案代价: Agent 所有任务都尝试编译执行, Agent 执行时间恶化. 还有可能因为环境依赖引发其他问题. 同时还会影响其他场景, 并不是所有的代码都可以测试, 比如 bash 脚本, html 代码.
- Bad Case 3: Agent 生成的代码使用了不安全的函数
 - 解决方案: 在系统提示词里禁止使用 memset 等不安全的函数, 或者生成完代码引导 Agent 做安全检查.
 - 方案代价: Agent 因噎废食, 导致 memset 的一些合法场景被误判. 因为 Agent 要做检查, 耗时与 token 消耗飙升.



挑战二：试错成本高



如同混沌系统，Coding Agent系统中任何微小的改动，都可能引发非线性的连锁反应，形成难以预测的“蝴蝶效应”，使得每一次试错都伴随着极高的验证代价。

核心原因：复杂性与评估难

非线性系统关联：集成LLM、规划器等异构组件，组件间依赖紧密。微小改动易引发不可控的全局影响，难以溯源。

多维评估的盲区：功能、质量、鲁棒性等维度难以通过离线测试穷尽。深层问题往往仅在真实生产环境处理任务时才暴露。

连锁后果：成本与效率的双重损耗

迭代放缓

详尽测试拉长周期，版本更新节奏被严重制约。

人力剧增

大量精力投入测试、审查与监控，团队负荷极大。

机会错失

漫长开发周期导致产品滞后，错过市场最佳窗口期。

挑战三：数据利用困难



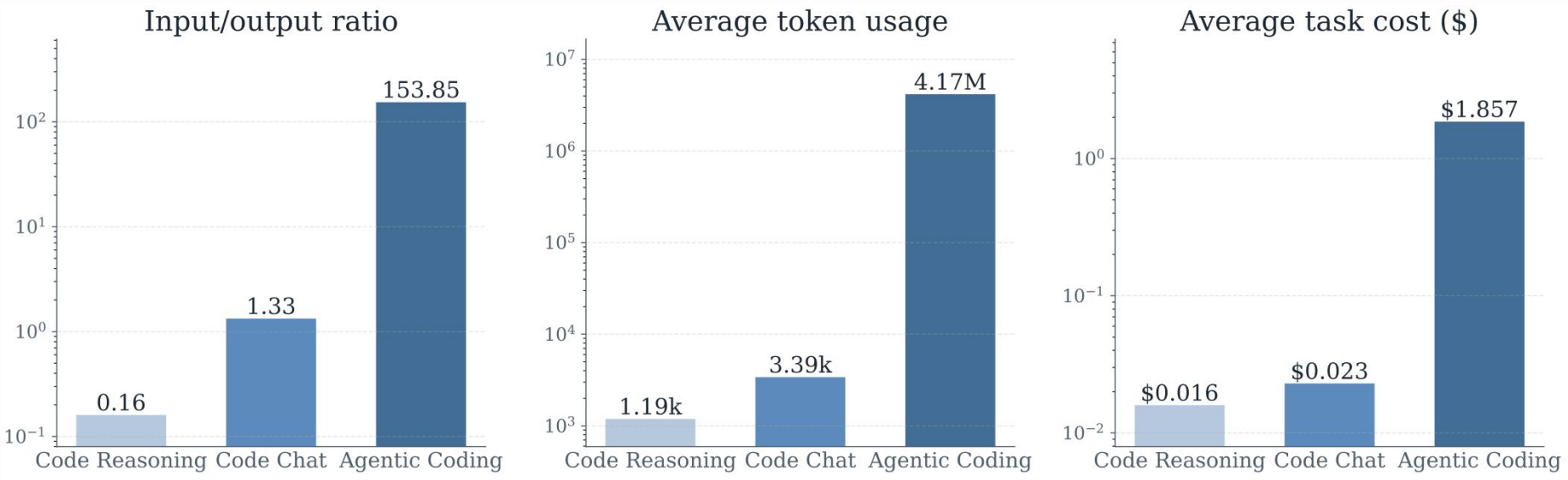
弱反馈信号≈无监督噪声

用户交互数据看似量大，实则大多是颗粒度极粗的弱信号——补全被接受、生成被点踩、任务最终失败。这些信号没有归因：用户拒绝一段代码，可能是命名风格不符、缩进不对、逻辑错误、不符合业务上下文，甚至只是当时改了需求。单一的“接受 / 拒绝”无法反哺具体哪一步错了、该怎么改，终局反馈支撑不起过程优化。

无标注轨迹无法直接复用

Agent 的执行日志虽然完整，但失败案例没有错误归因，成功案例也无法识别关键决策点。没有人工或强规则标注的轨迹，对模型训练而言本质是噪声，无法直接用于提升决策准确率。

挑战四: 推理成本高

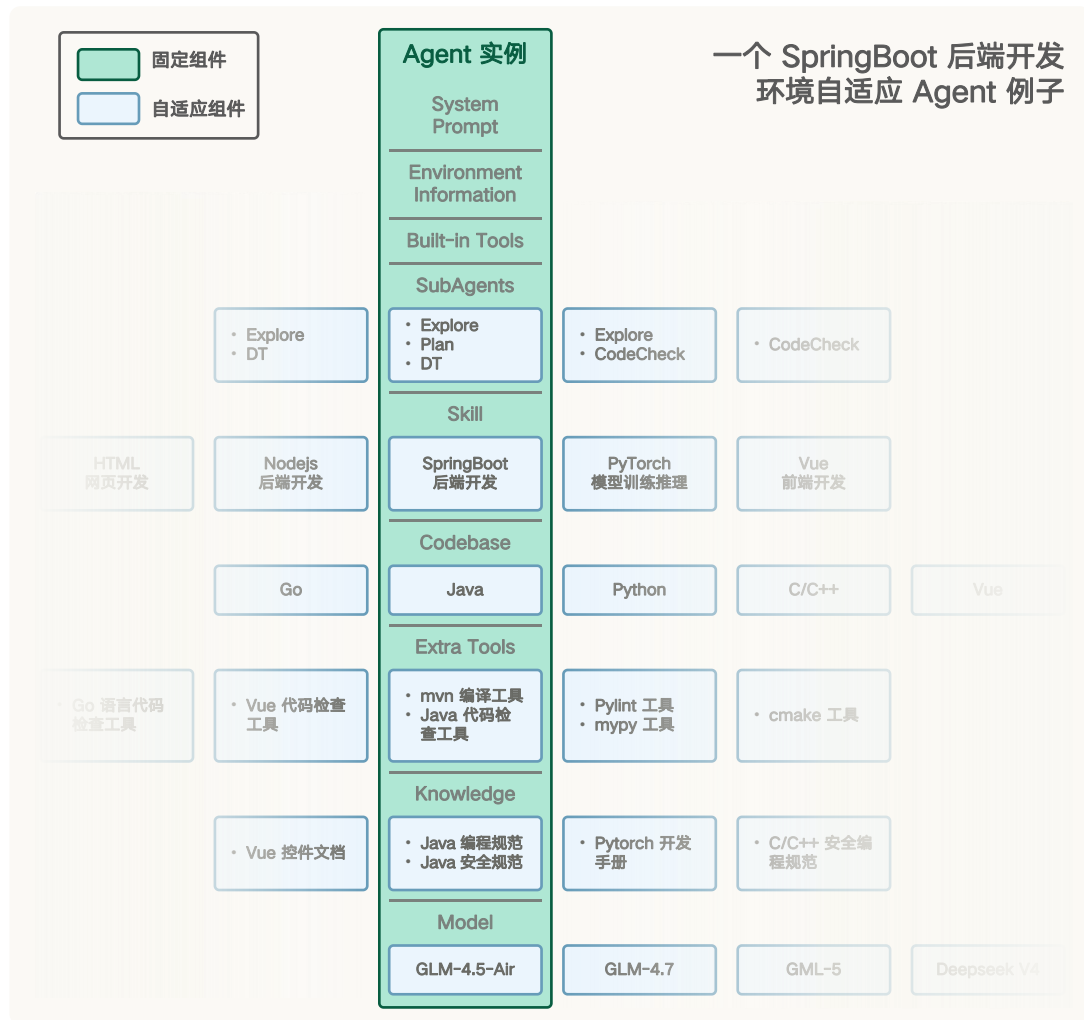


数据来源:
L. Bai *et al.*, "How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks," arXiv preprint arXiv:2604.22750, 2026.

任务类型	典型 Token 区间	备注
简单补全 / 解释报错	300 ~ 2,000	Copilot 日常使用
单函数 / 小功能开发	2,000 ~ 20,000	Copilot Chat / 轻量 Agent
多文件模块 / 中等重构	20,000 ~ 100,000	Cursor / Claude Code 常规任务
复杂重构 / 多模块改造	100,000 ~ 400,000	Claude Code 重度任务
老系统理解 + 架构级修改	200 万 ~ 800 万	Agent 全链路任务（接近 SWE-bench）

04 解构华为云码道

环境自适应 Agent



Coding Agent 优化过程中的跷跷板效应:

在 Agent 迭代过程中, 针对某一核心能力维度的专项优化动作, 会不可避免地引发另一个 / 多个强关联的核心能力指标出现显著反向衰减, 二者呈现此消彼长的负相关制衡关系.

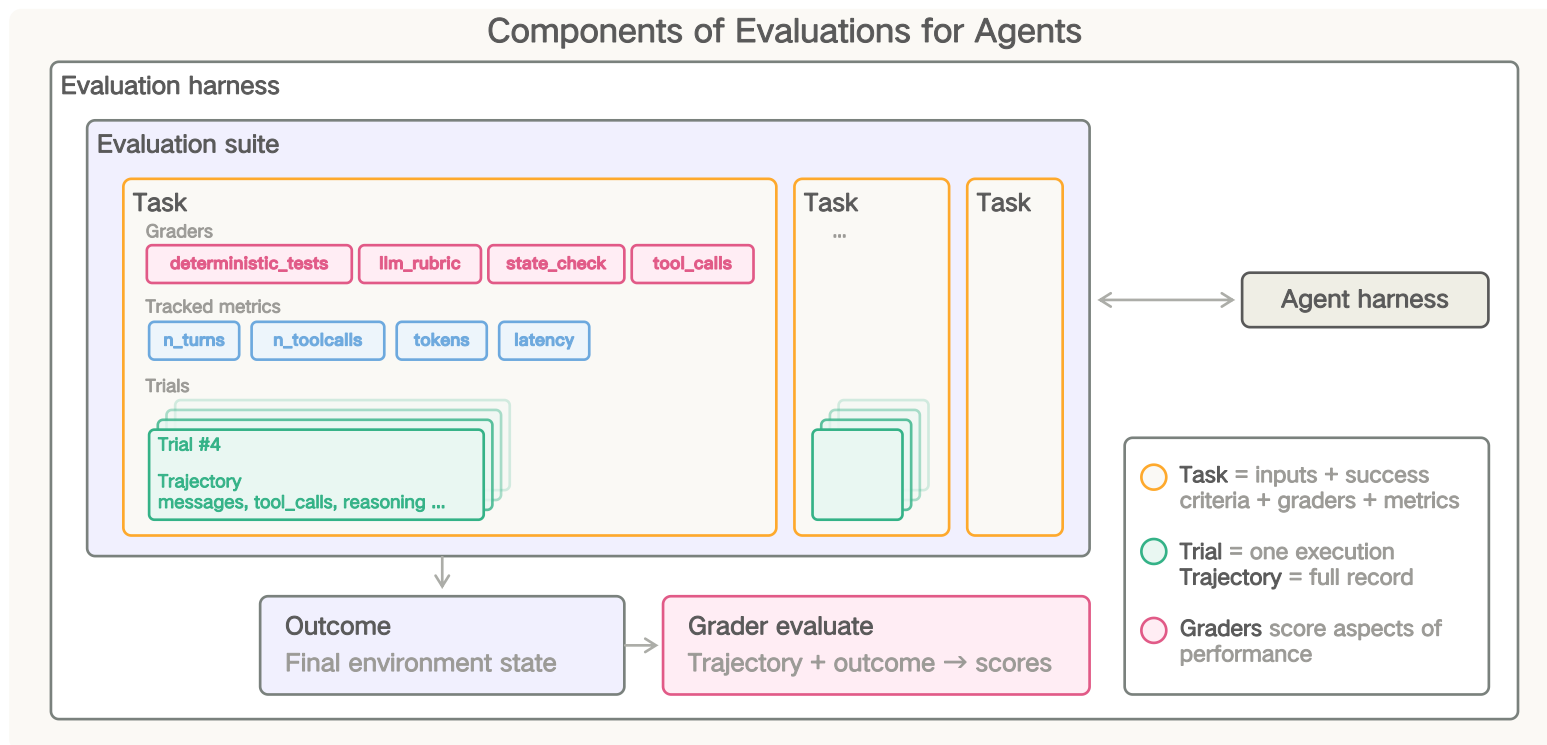
跷跷板效应的根因:

- 核心指标的冲突: 比如极致的正确性需要更多校验, 必然牺牲效率; 极致的专项适配必然牺牲通用泛化能力。
- 模型注意力限制: 大模型的上下文注意力、参数容量存在上限, Prompt 中增加的约束 (安全、规范、校验规则) 会稀释核心功能的注意力分配; 专项微调会让参数分布向特定场景倾斜, 引发通用知识的灾难性遗忘。

解决方案:

- 模型能力提升:** 模型能力提升的核心目标是减少"为补短板而牺牲长板"的必要性——如果模型单轮生成就能兼顾正确性、效率、泛化性, 就无需通过多轮校验牺牲效率, 也无需通过专项微调牺牲泛化性。
- 环境自适应 Agent:** Agent 不是固定一成不变的, 而是根据用户环境信息以及场特征动态变化的. 比如左图, 插件根据当前项目的特征, 检测到用户是一个大型的 Springboot 后端项目:

Agent 能力评估



评分器 Grader

确定性评分器(deterministic-grader)对于 CodeAgent 非常适合, 可以很容易判断代码是否能编译, 是否能通过测试用例. 确定性 Grader 是对 Outcome 进行评分.

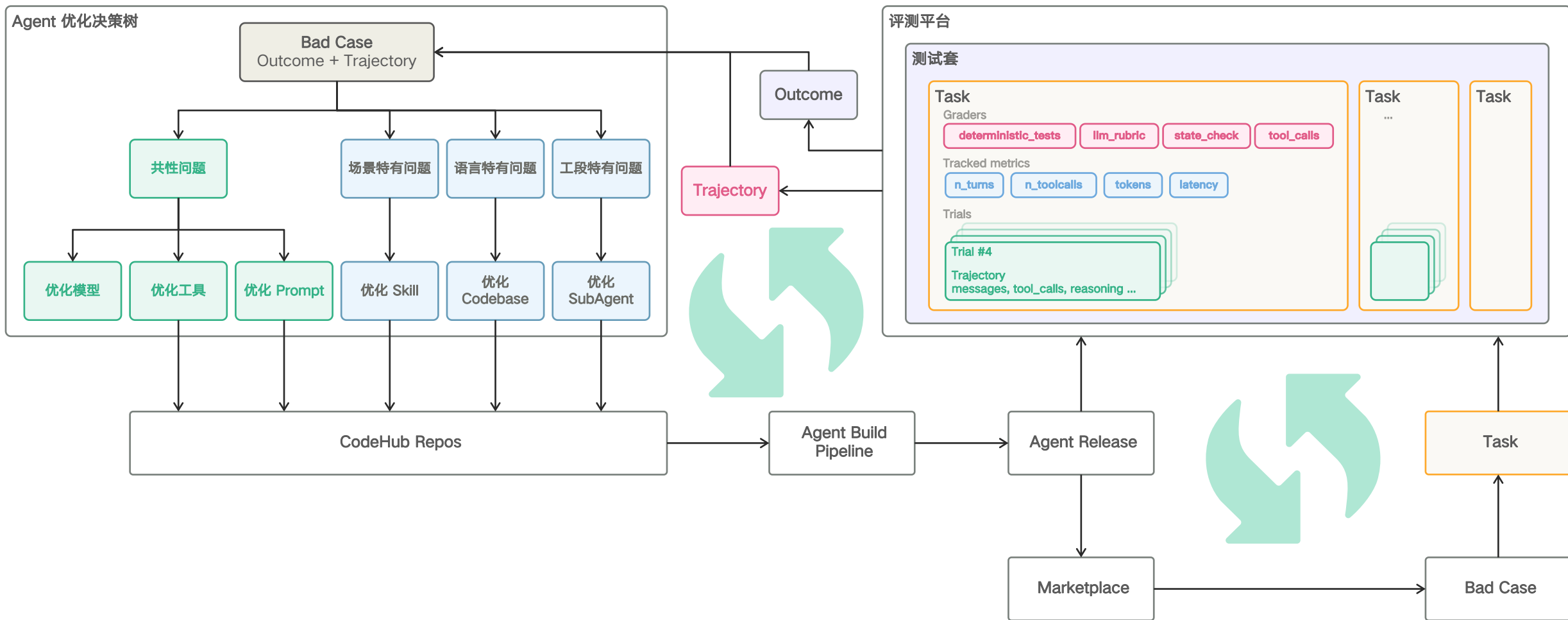
但是仅仅对 Outcome 进行评分是不够的, 比如代码生成质量好坏, 代码仓理解是否正确, 调用的工具是否合理, 消耗的 Token 是否必要, 都无法判断.

因此, 还需要有**非确定性的评分器**, 比如 llm_rubric, tool_calls, static_analysis, 对 Agent 的执行过程进行打分.

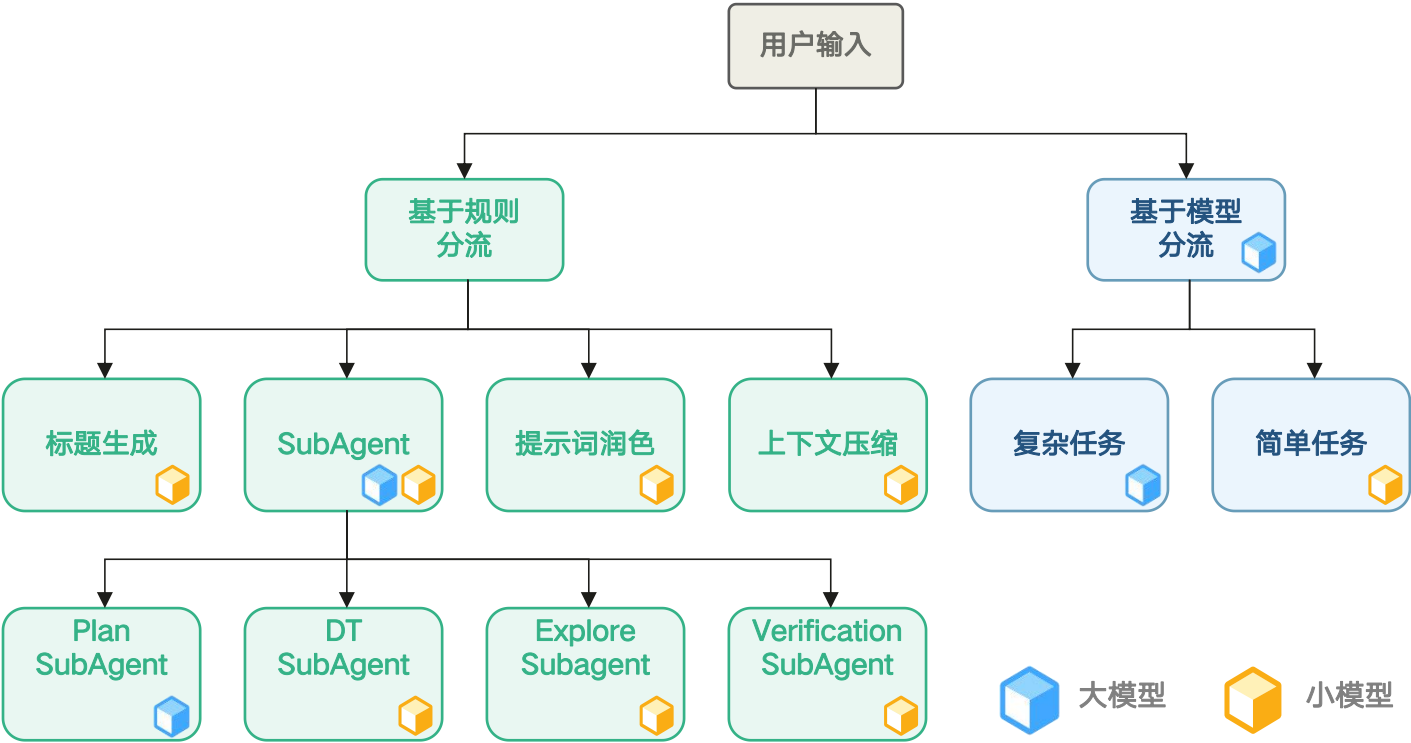
某个 Agent 在某个任务的最终分数计算方式如下:

$$\text{最终分数} = \text{deterministic}(\text{outcome}) + \text{non-deterministic}(\text{trajectory})$$

评估和运营双循环



大小模型协同



方案	一次通过率	Token 消耗	
纯大模型	62/128	GLM-4.7 358B	178,615,868
基于模型分流	61/128	Qwen-3.6 27B	47,555,306
		GLM-4.7 358B	122,177,175
基于模型分流	60/128	Qwen-3.6 27B	43,273,457
		GLM-4.7 358B	134,558,212

- 评测验证, 精度掉点 $(62 - 61.5) / 128 = 0.4\%$
- Token 消耗 $((122,177,175 + 47,555,306 / 8) + (134,558,212 + 43,273,457 / 8)) / 2 = 134,044,491$
- Token 成本节约 $(178,615,868 - 134,044,491) / 178,615,868 = 25.0\%$

工具优化: 提升 Agent 执行效率以及效果

Claude Code

1 次模型请求, 消耗 40+ token

role: Assistant

Grep(pattern: "getItem\\(", output_mode: "content")

role: Tool

SysUserApi.java

```
130-> public CommonResult<> getItem(@PathVariable Long id) {  
131->     UmsAdmin admin = adminService.getById(id);  
132->     return CommonResult.success(admin);  
133-> }
```

CodeAgent

2 次模型请求, 消耗 300+ token

role: Assistant

search_file(pattern: "getItem\\(")

role: Tool

SysUserApi.java:130

role: Assistant

read_file(path: "SysUserApi.java")

role: Tool

```
...  
public CommonResult<> getItem(@PathVariable Long id) {  
    UmsAdmin admin = adminService.getById(id);  
    return CommonResult.success(admin);  
}  
...
```

高效的工具, 可以

1. 减少工具执行轮次/模型交互轮次, 加速Agent执行
2. 减少 Token 消耗, 节约上下文, 避免频繁触发压缩
3. 减少无关信息对模型的干扰, 生成更准确

工具优化: 一切为了模型

- **高效跨平台:** 都是基于 ripgrep, 而不是传统 grep/find, 跨平台、速度更快、默认更适合代码库搜索。
- **入参标准化:** 有统一输入 schema, 避免模型乱传参数。
- **眼不见为净:** 自动带 --hidden, 但同时排除 .git/.svn/.hg/.bzz/.jj/.sl 这类噪声目录。
- **上下文保护:** 自动加 --max-columns 500, 避免 base64、压缩文件、minified JS 把上下文冲爆。支持 head_limit + offset 分页, 不让一次搜索塞满上下文。
- **多模式输出:** 支持 glob、type、content/files_with_matches/count 三种输出模式。
- **上下文节约:** 搜索结果统一改成相对路径, 降低上下文占用。
- **智能重排序:** 对只返回文件名的模式, 会再按文件修改时间排序, 让“最近改过的文件”优先出现, 更符合 agent 工作流。
- **不该看不看:** 会应用权限系统中的“读禁止规则”, 把不该看的文件直接从搜索层面隐藏掉。

Grep

- **防止 Agent 偷懒:** Agent 读都没读, 不准改。
- **防止 Agent 落后:** Agent 读了, 但是文件被改了, 还是不准改。
- **防止 Agent 如改:** 当 old_string 和 new_string 相同时, 报错。
- **防止 Agent 大改:** 要求 old_string 是唯一的, 如果命中了多次, 必须显示的 replace_all=true, 否则拒绝。
- **防止 Agent 乱改:** 保留文件原有编码和换行风格, Edit 读文件时就检测编码和行尾, 写回时按原格式保存。
- **防止 Agent 白改:** 模型不善于誊写, 容易抄错, Edit 对模型输出做纠偏, 防止找不到。
- **防止 Agent 瞎改:** 会先走写权限系统, 检查 deny/ask/allow 规则、工作目录范围。
- **防止 Agent 错改:** 改完会调用 LSP 做代码检查, 如果有问题第一时间通知模型修改。

Edit

- **路径归一化,**防止 ~、相对路径、Windows 分隔符差异绕过权限
- **返回的是结构化 tool_result,** 文本会带行号。
- **相同文件、相同行区间、mtime 没变时,** 直接返回 file_unchanged stub, 避免重复塞上下文。
- **会在读完后估 token,** 必要时调用 API 精确计数, 超过上限直接报错。

Read

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



05 总结与展望

■ 总结

- 从 AI 辅助研发到辅助 AI 研发
- 环境自适应机制 -- 解决跷跷板效应
- 自动化评测平台 -- 快速试错, 迭代产品
- 评估运营双循环 -- 持续提升 Agent 能力
- 大中小模型协同 -- 降低推理成本
- 工具集能力优化 -- 提升 Agent 综合能力

AI 编程工具从 Copilot 向智能体团队演进

Copilot
2023 -- 2024

面向代码



32K

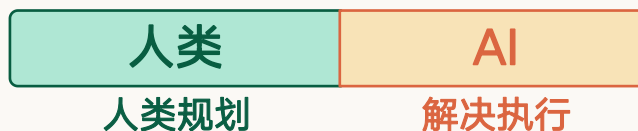
一问一答，输出有限，手动采纳，幻觉严重



Copilot模式
代码补全与研发问答

Agent
2024 -- 2026

面向任务

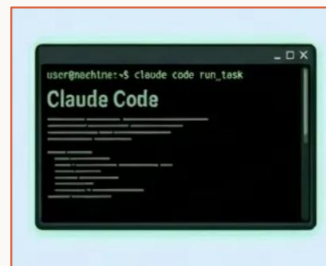


200K

多轮迭代，自主规划，工具使用，自动验证



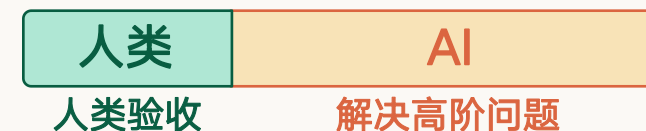
Agent GUI 模式
通过 Agent 自主规划完成任务



Agent CLI 模式

Agentic Team
2026 -- Future

面向目标



1M+

目标拆解，并行交叉探索，自管理，自演进



Agent Team
Agent 指挥 Agent

GitHub Copilot/GPT4

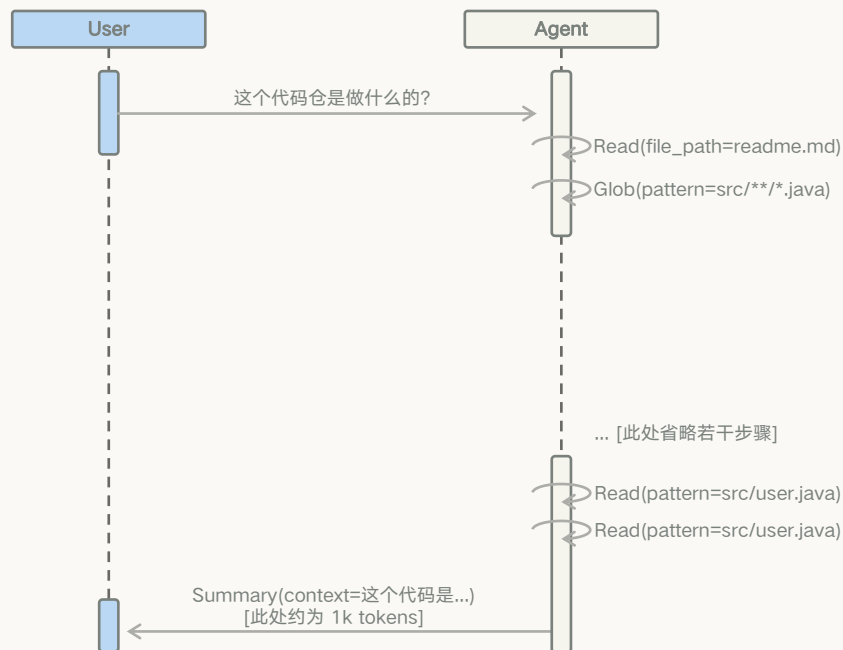
Cursor Composer/Sonnet 3.5

Claude Code/Opus 4

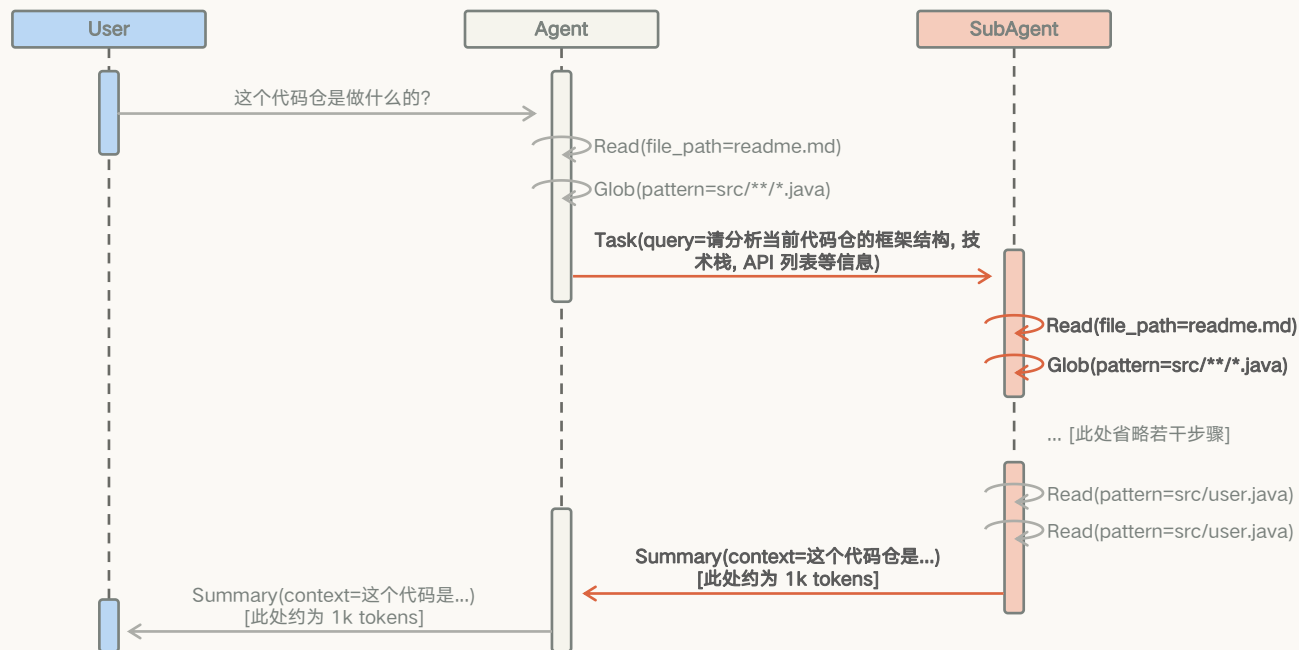
Claude Agent Team/Opus4.6

SubAgent or not SubAgent?

Agent 处理任务时序图 (无 SubAgent)



Agent 处理任务时序图 (有 SubAgent)



以下情况考虑使用 SubAgent:

- 主 Agent 上下文窗口不足, 需要将 SubAgent 的上下文与主 Agent 隔离.
- 需要强制约束 SubAgent 可以使用的工具集合
- 针对特定领域设置 SubAgent 针对性提示词(Skill 无法满足)
- 需要将任务分配到更快、更便宜的模型来控制成本或者加速推理

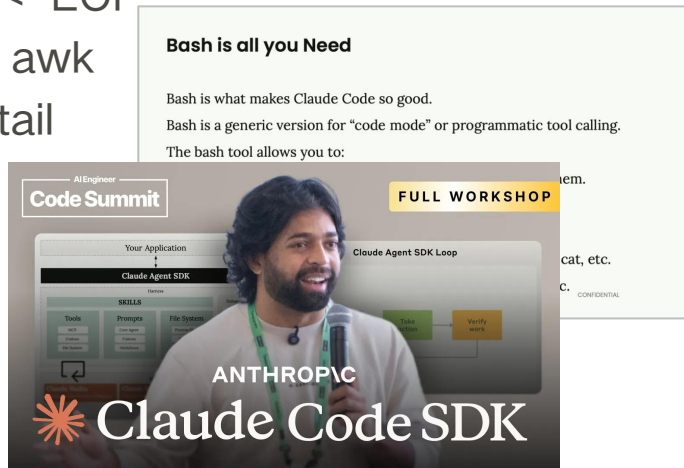
以下情况慎重使用 SubAgent:

- 简单任务: 主 Agent 的上下文空间可以处理.
- 完全委派: 主 Agent 直接将任务委派给 SubAgent.
- 上下文依赖: SubAgent 和主 Agent 之间存在上下文依赖.

Bash is All You Need?

Bash 是一个终极工具, Agent 可以用 Bash 做任何事情

- 写文件: `cat > file << 'EOF'`
- 编辑文件: `sed` 或者 `awk`
- 读文件: `cat`, `head`, `tail`
- 删除文件: `rm`
- 查找文件: `find`, `ls`
- 查找内容: `grep`
- 网络请求: `curl`
- 版本管理: `git`
-



从图灵完备角度看: 给 Agent Bash 工具相当于给它整个操作系统的能力。典型例子: [mini-swe-agent](#)。

除此之外, Bash 工具还有一个巨大优势: 模型天生会用 Bash。

Anthropic 的研究员 Thariq 曾经也提过: Bash is All You Need。

Claude Code 系统提示词里有这么一段话, 告诉模型: 能不用 Bash 就不用 Bash。

Use specialized tools instead of bash commands when possible, **as this provides a better user experience**. For file operations, use dedicated tools:

- **Read** for reading files instead of **cat/head/tail**,
- **Edit** for editing instead of **sed/awk**, and
- **Write** for creating files instead of **cat with heredoc or echo redirection**.

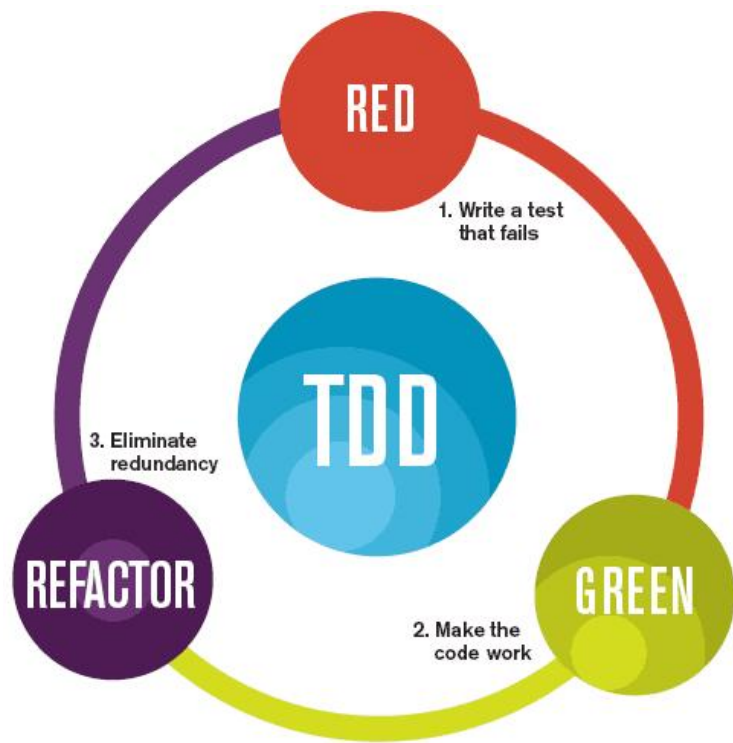
核心理由是: 专用工具在安全性、易用性、鲁棒性上, 通常远胜于通过 Bash 命令 (如 `sed/awk/cat`) 进行的黑魔法操作

Bad Case 1: 通过 `cat` 读了一个敏感文件。安全

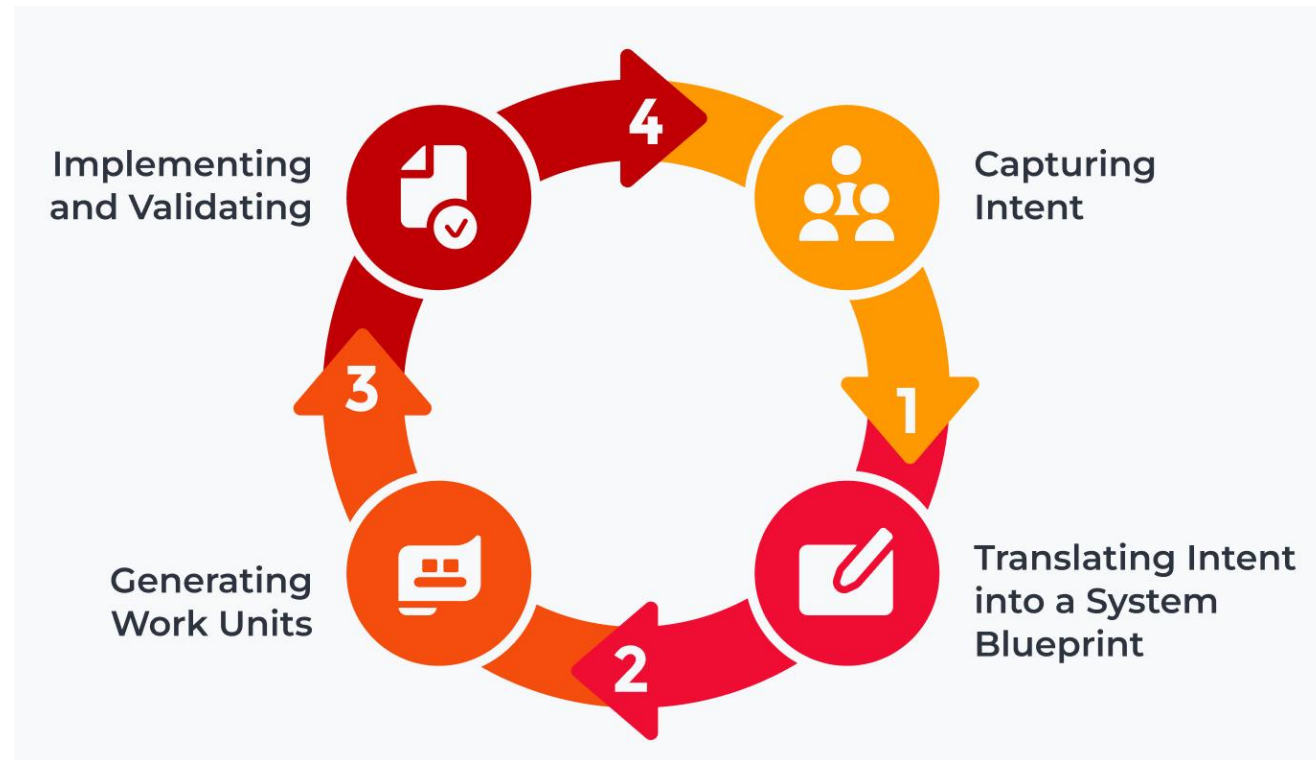
Bad Case 2: 通过 `sed` 编辑文件, 但是由于没能完全匹配而失败, 并且没有任何提示。易用 鲁棒

Bad Case 3: 通过 `cat > code.java << 'EOF' ...` 来写入文件, 模型要处理大量的转义字符, 非常容易出错。易用

SDD or TDD?



The mantra of Test-Driven Development (TDD) is "red, green, refactor."



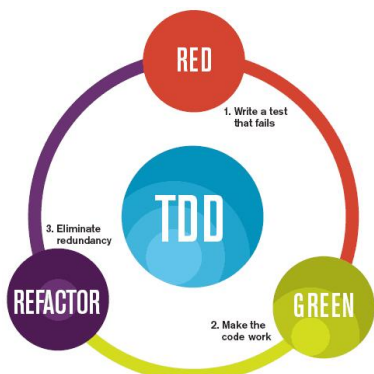
SDD or TDD?

01. TDD (测试驱动开发)

以可执行的测试用例为核心契约，遵循「红→绿→重构」循环，让Agent围绕测试目标生成代码，以测试通过率作为功能完成的核心标准。

核心优势

建立零歧义验收标准，倒逼接口边界清晰；天然适配Agent分步执行，是重构与迭代的可靠安全网。



The mantra of Test-Driven Development (TDD) is "red, green, refactor."

核心逻辑：通过“先定义失败，再实现成功”的闭环，确保每一行代码都有明确的验证目标。适合算法模块、工具函数等逻辑密集型场景。

潜在局限

仅保障局部逻辑正确，无法约束全局业务意图；测试质量决定代码上限，易催生“为过测试而编码”的劣化实现。

02. SDD (规格驱动开发)

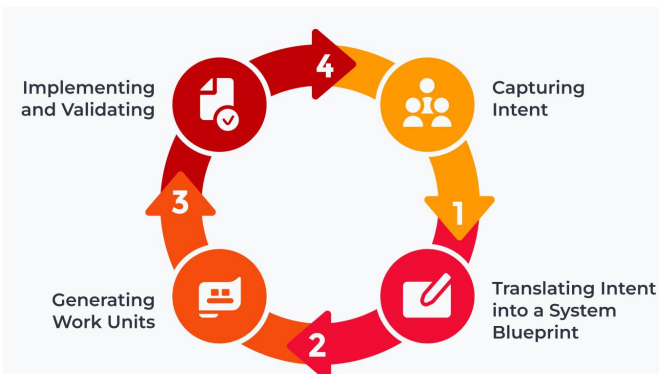
编码前先输出结构化规格文档（含业务需求、接口契约、架构约束），以规格作为Agent编码的唯一真值输入，从源头约束输出边界。

核心优势

从源头抑制架构漂移，结构化输入天然适配AI理解；实现文档与代码同步产出，便于跨会话持久化项目上下文。

潜在局限

前置投入成本高，且属于纸面约束无运行时强制力；存在“规格漂移”风险，无法替代底层逻辑的正确性验证。



“先定义规则，再生成代码”的模式，具有高度的一致性。适合中大型系统、多场景。



单击此处添加标题



Sadipscing eu at dolore
magna voluptua magna
nisl lorem.

Click to add text



Sadipscing eu at dolore
magna voluptua magna
nisl lorem.

Click to add text



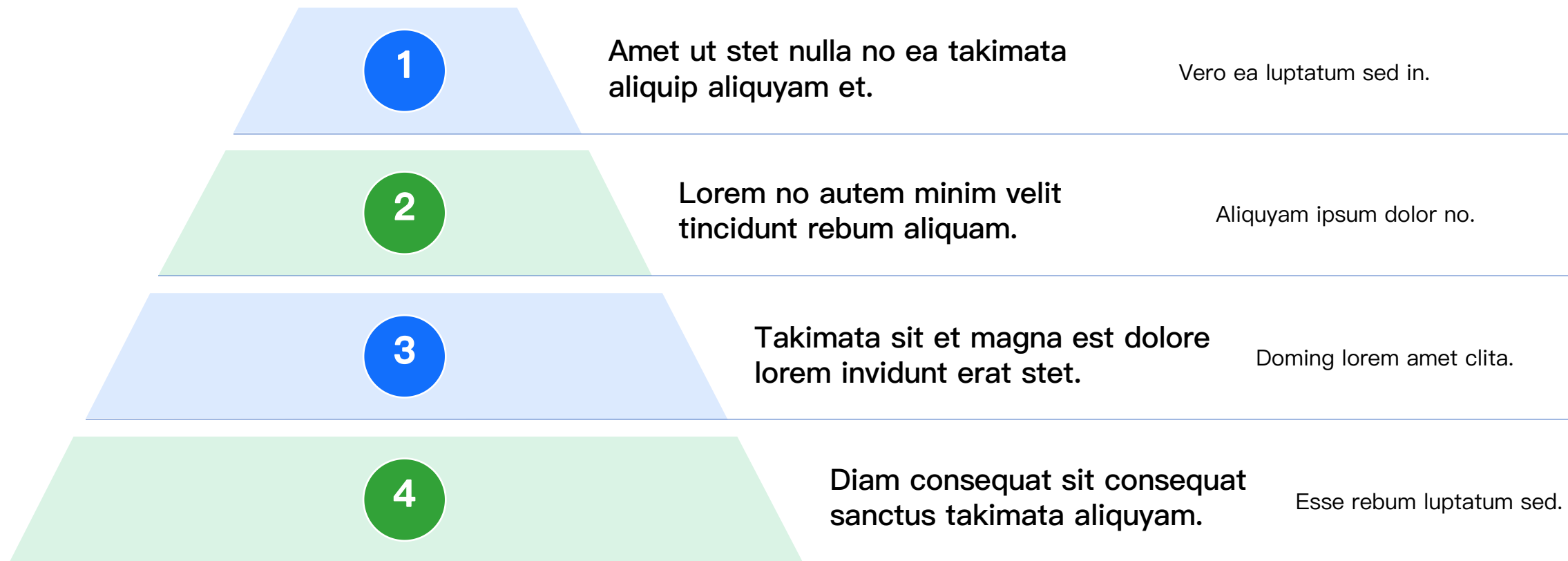
Sadipscing eu at dolore
magna voluptua magna
nisl lorem.

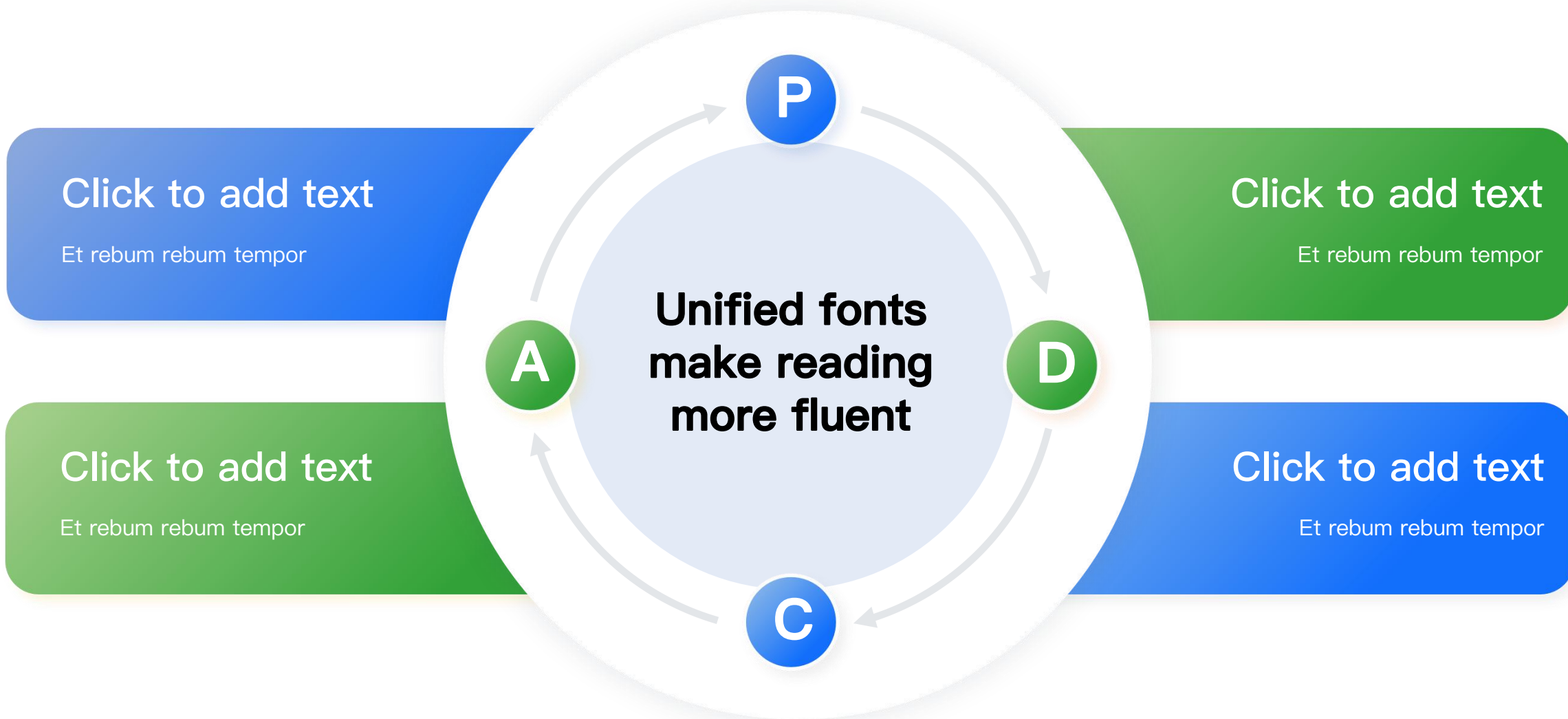
Click to add text



Sadipscing eu at dolore
magna voluptua magna
nisl lorem.

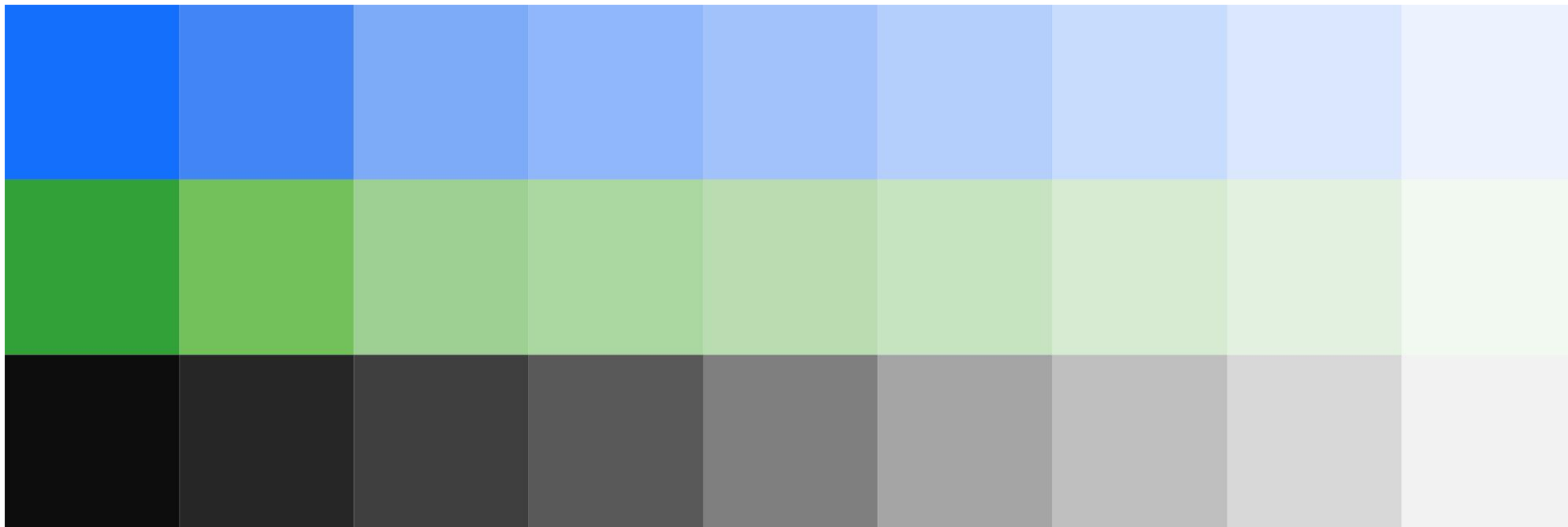
Click to add text







颜色规范



字体规范（内容页）

一级标题 苹方字体 36号字 中粗体 黑色

二级标题 苹方字体 18号字 中黑体 蓝色

三级标题 苹方字体 16号字 常规体 黑色

正文 苹方字体 12-14号字 常规体 黑色淡色 25%

正文 段落间距 多倍行距 1.3 段后间距 12

间距:

之前:

0 pt

行距:

之后:

12 pt

多倍行距

在:

1.3



内容页示意

二级标题 18号字 苹方字体 中黑体

QCon 全球软件开发大会是由极客邦科技旗下 InfoQ 中国主办的综合性技术盛会，自 2007 年 3 月份开始举办以来，已经有超万名有多年从业经验的技术人员参加过 Qcon 大会。

正文少时 建议 14 号字

二级标题 18号字 苹方字体 中黑体

QCon 全球软件开发大会是由极客邦科技旗下 InfoQ 中国主办的综合性技术盛会，自 2007 年 3 月份开始举办以来，已经有超万名有多年从业经验的技术人员参加过 Qcon 大会。

QCon 全球软件开发大会是由极客邦科技旗下 InfoQ 中国主办的综合性技术盛会，自 2007 年 3 月份开始举办以来，已经有超万名有多年从业经验的技术人员参加过 Qcon 大会。

正文多时 建议 12 号字

 单击此处添加标题

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会